

第26章 串行 FLASH 文件系统 FatFs

26.1 文件系统

即使读者可能不了解文件系统，读者也一定对“文件”这个概念十分熟悉。数据在 PC 上是以文件的形式储存在磁盘中的，这些数据的形式一般为 ASCII 码或二进制形式。在上一章我们已经写好了 SPI Flash 芯片的驱动函数，我们可以非常方便的在 SPI Flash 芯片上读写数据。如需要记录本书的书名“零死角玩转 STM32-F407 系列”，可以把这些文字转化成 ASCII 码，存储在数组中，然后调用 SPI_FLASH_BufferWrite 函数，把数组内容写入到 SPI Flash 芯片的指定地址上，在需要的时候从该地址把数据读取出来，再对读出来的数据以 ASCII 码的格式进行解读。

但是，这样直接存储数据会带来极大的不便，如难以记录有效数据的位置，难以确定存储介质的剩余空间，以及应以何种格式来解读数据。就如同一个巨大的图书馆无人管理，杂乱无章地存放着各种书籍，难以查找所需的文档。想象一下图书馆的采购人员购书后，把书籍往馆内一扔，拍拍屁股走人，当有人来借阅某本书的时候，就不得不一本本地查找。这样直接存储数据的方式对于小容量的存储介质如 EEPROM 还可以接受，但对于 SPI Flash 芯片或者 SD 卡之类的大容量设备，我们需要一种高效的方式来管理它的存储内容。

这些管理方式即为文件系统，它是为了存储和管理数据，而在存储介质建立的一种组织结构，这些结构包括操作系统引导区、目录和文件。常见的 windows 下的文件系统格式包括 FAT32、NTFS、exFAT。在使用文件系统前，要先对存储介质进行格式化。格式化先擦除原来内容，在存储介质上新建一个文件分配表和目录。这样，文件系统就可以记录数据存放的物理地址，剩余空间。

使用文件系统时，数据都以文件的形式存储。写入新文件时，先在目录中创建一个文件索引，它指示了文件存放的物理地址，再把数据存储到该地址中。当需要读取数据时，可以从目录中找到该文件的索引，进而在相应的地址中读取数据。具体还涉及到逻辑地址、簇大小、不连续存储等一系列辅助结构或处理过程。

文件系统的存在使我们在存取数据时，不再是简单地向某物理地址直接读写，而是要遵循它的读写格式。如经过逻辑转换，一个完整的文件可能被分开成多段存储到不连续的物理地址，使用目录或链表的方式来获知下一段的位置。

上一章的 SPI Flash 芯片驱动只完成了向物理地址写入数据的工作，而根据文件系统格式的 logic 转换部分则需要额外的代码来完成。实质上，这个逻辑转换部分可以理解为当我们需要写入一段数据时，由它来求解向什么物理地址写入数据、以什么格式写入及写入一些原始数据以外的信息(如目录)。这个逻辑转换部分代码我们也习惯称之为文件系统。

26.2 FatFs 文件系统简介

上面提到的逻辑转换部分代码(文件系统)即为本章的要点，文件系统庞大而复杂，它需要根据应用的文件系统格式而编写，而且一般与驱动层分离开来，很方便移植，所以工程应用中一般是移植现成的文件系统源码。

FatFs 是面向小型嵌入式系统的一种通用的 FAT 文件系统。它完全是由 AISI C 语言编写并且完全独立于底层的 I/O 介质。因此它可以很容易地不加修改地移植到其他的处理器当中，如 8051、PIC、AVR、SH、Z80、H8、ARM 等。FatFs 支持 FAT12、FAT16、FAT32 等格式，所以我们利用前面写好的 SPI Flash 芯片驱动，把 FatFs 文件系统代码移植到工程之中，就可以利用文件系统的各种函数，对 SPI Flash 芯片以“文件”格式进行读写操作了。

FatFs 文件系统的源码可以从 fatfs 官网下载：

http://elm-chan.org/fsw/ff/00index_e.html

26.2.1 FatFs 的目录结构

在移植 FatFs 文件系统到开发板之前，我们先要到 FatFs 的官网获取源码，最新版本为 R0.11a，官网有对 FatFs 做详细的介绍，有兴趣可以了解。解压之后可看到里面有 doc 和 src 这两个文件夹，见图 26-1。doc 文件夹里面是一些使用帮助文档；src 才是 FatFs 文件系统的源码。

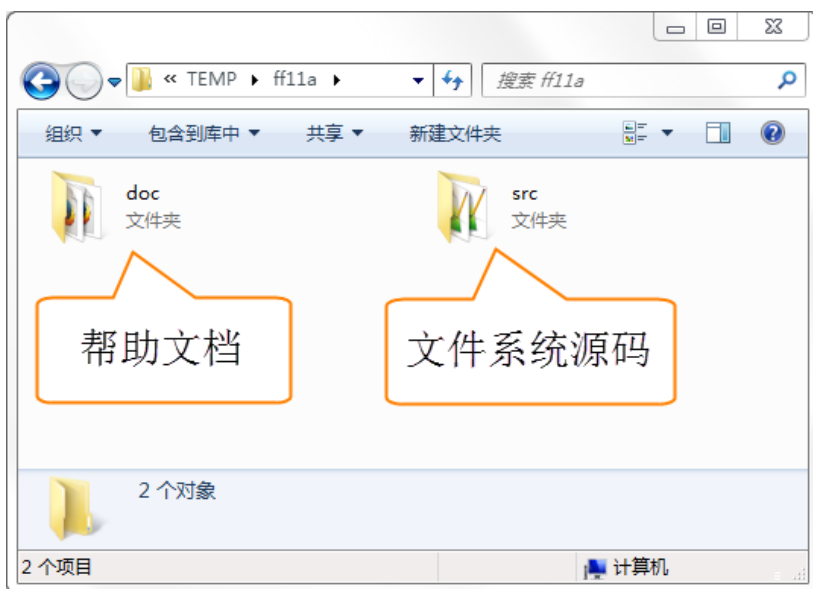


图 26-1 FatFs 文件目录

26.2.2 FatFs 帮助文档

打开 doc 文件夹，可看到如图 26-2 的文件目录：

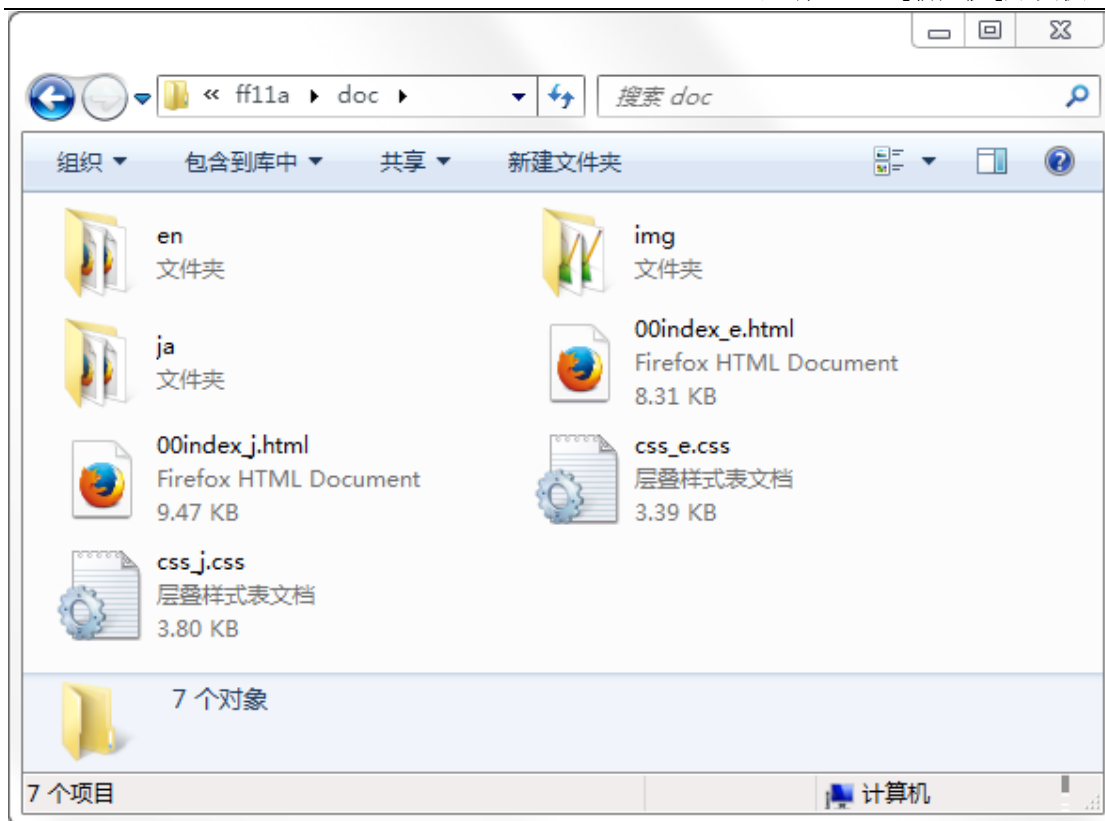


图 26-2 doc 文件夹的文件目录

其中 en 和 ja 这两个文件夹里面是编译好的 html 文档，讲的是 FATFS 里面各个函数的使用方法，这些函数都是封装得非常好的函数，利用这些函数我们就可以操作 SPI Flash 芯片。有关具体的函数我们在用到的时候再讲解。这两个文件夹的唯一区别就是 en 文件夹下的文档是英文的，ja 文件夹下的是日文的。img 文件夹包含 en 和 ja 文件夹下文件需要用到的图片，还有四个名为 app.c 文件，内容都是 FatFs 具体应用例程。00index_e.html 和 00index_j.html 是一些关于 FATFS 的简介，至于另外两个文件可以不看。

26.2.3 FATFS 源码

打开 src 文件夹，可看到如图 26-3 的文件目录：

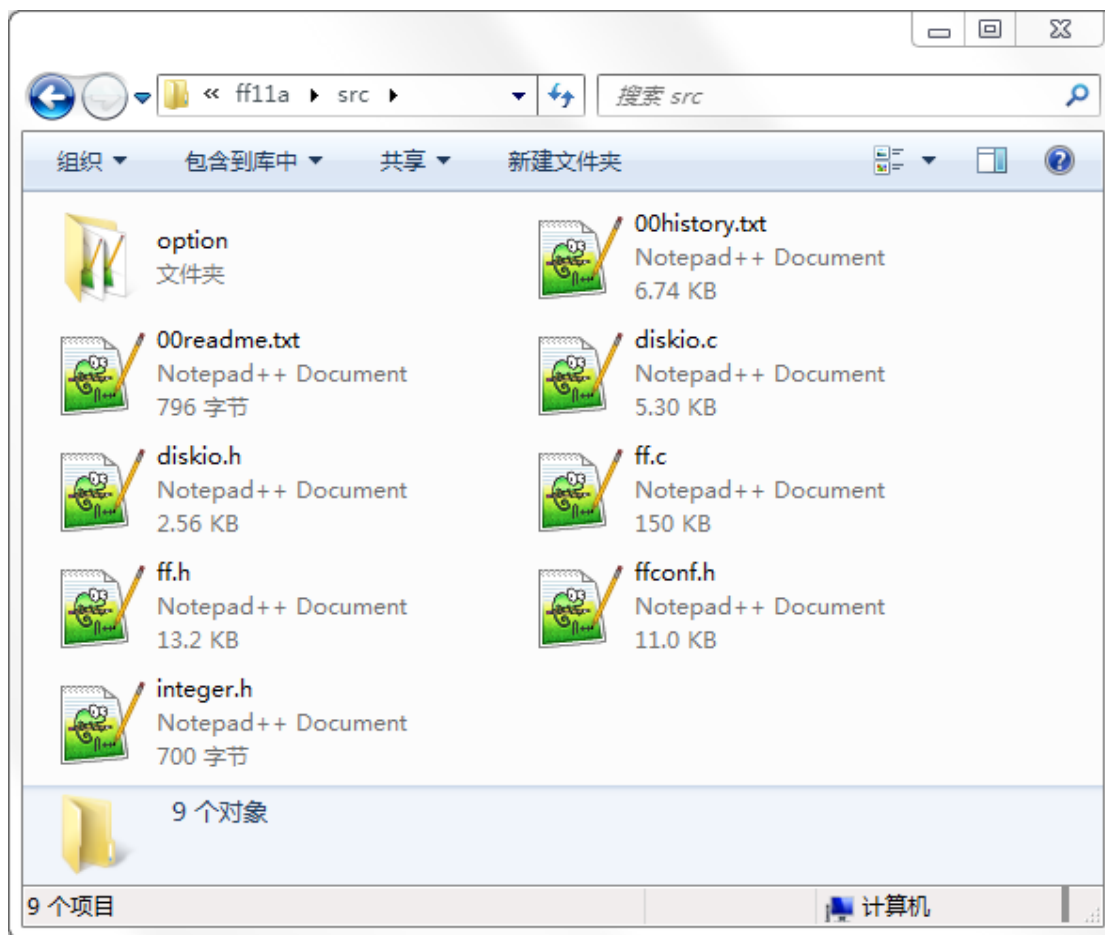


图 26-3 src 文件夹的文件目录

option 文件夹下是一些可选的外部 c 文件，包含了多语言支持需要用到的文件和转换函数。

diskio.c 文件是 FatFs 移植最关键的文件，它为文件系统提供了最底层的访问 SPI Flash 芯片的方法，FatFs 有且仅有它需要用到与 SPI Flash 芯片相关的函数。diskio.h 定义了 FatFs 用到的宏，以及 diskio.c 文件内与底层硬件接口相关的函数声明。

00history.txt 介绍了 FatFs 的版本更新情况。

00readme.txt 说明了当前目录下 diskio.c、diskio.h、ff.c、ff.h、integer.h 的功能。

src 文件夹下的源码文件功能简介如下：

- ❑ integer.h: 文件中包含了一些数值类型定义。
- ❑ diskio.c: 包含底层存储介质的操作函数，这些函数需要用户自己实现，主要添加底层驱动函数。
- ❑ ff.c: FatFs 核心文件，文件管理的实现方法。该文件独立于底层介质操作文件的函数，利用这些函数实现文件的读写。
- ❑ cc936.c: 本文件在 option 目录下，是简体中文支持所需要添加的文件，包含了简体中文的 GBK 和 Unicode 相互转换功能函数。

- ❑ `ffconf.h`: 这个头文件包含了对 FatFs 功能配置的宏定义，通过修改这些宏定义就可以裁剪 FatFs 的功能。如需要支持简体中文，需要把 `ffconf.h` 中的 `_CODE_PAGE` 的宏改成 936 并把上面的 `cc936.c` 文件加入到工程之中。

建议阅读这些源码的顺序为：`integer.h` --> `diskio.c` --> `ff.c`。

阅读文件系统源码 `ff.c` 文件需要一定的功底，建议读者先阅读 FAT32 的文件格式，再去分析 `ff.c` 文件。若仅为使用文件系统，则只需要理解 `integer.h` 及 `diskio.c` 文件并会调用 `ff.c` 文件中的函数就可以了。本章主要讲解如何把 FATFS 文件系统移植到开发板上，并编写一个简单读写操作范例。

26.3 FatFs 文件系统移植实验

26.3.1 FatFs 程序结构图

移植 FatFs 之前我们先通过 FatFs 的程序结构图了解 FatFs 在程序中的关系网络，见图 26-4。

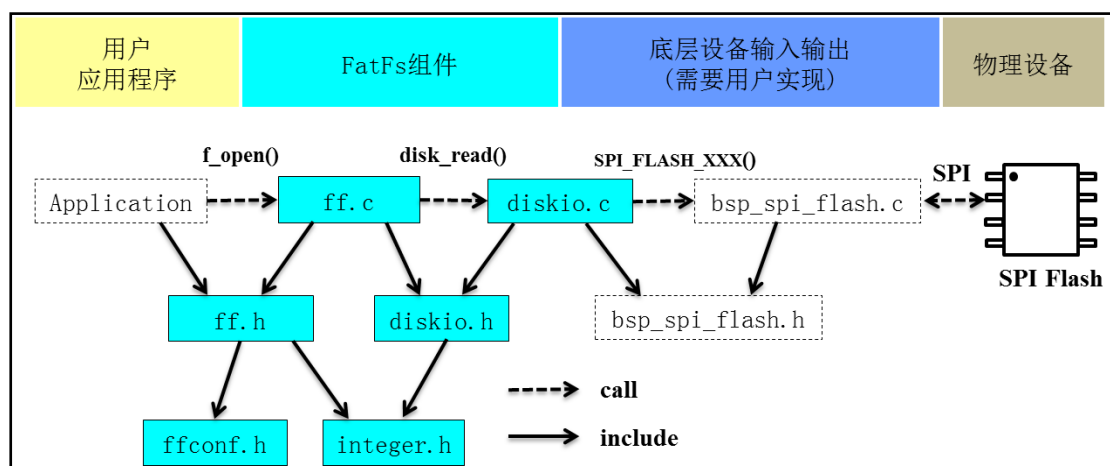


图 26-4 FatFs 程序结构图

用户应用程序需要由用户编写，想实现什么功能就编写什么的程序，一般我们只用到 `f_mount()`、`f_open()`、`f_write()`、`f_read()` 就可以实现文件的读写操作。

FatFs 组件是 FatFs 的主体，文件都在源码 `src` 文件夹中，其中 `ff.c`、`ff.h`、`integer.h` 以及 `diskio.h` 四个文件我们不需要改动，只需要修改 `ffconf.h` 和 `diskio.c` 两个文件。

底层设备输入输出要求实现存储设备的读写操作函数、存储设备信息获取函数等等。我们使用 SPI Flash 芯片作为物理设备，在上一章节已经编写好了 SPI Flash 芯片的驱动程序，这里我们就直接使用。

26.3.2 硬件设计

FatFs 属于软件组件，不需要附带其他硬件电路。我们使用 SPI Flash 芯片作为物理存储设备，其硬件电路在上一章已经做了分析，这里就直接使用。

26.3.3 FatFs 移植步骤

上一章我们已经实现了 SPI Flash 芯片驱动程序，并实现了读写测试，为移植 FatFs 方便，我们直接拷贝一份工程，我们在工程基础上添加 FatFs 组件，并修改 main 函数的用户程序即可。

- 1) 先拷贝一份 SPI Flash 芯片测试的工程文件(整个文件夹)，并修改文件夹名为“SPI—FatFs 文件系统”。将 FatFs 源码中的 src 文件夹整个文件夹拷贝一份至“SPI—FatFs 文件系统\USER\”文件夹下并修改名为“FATFS”，见图 26-5。

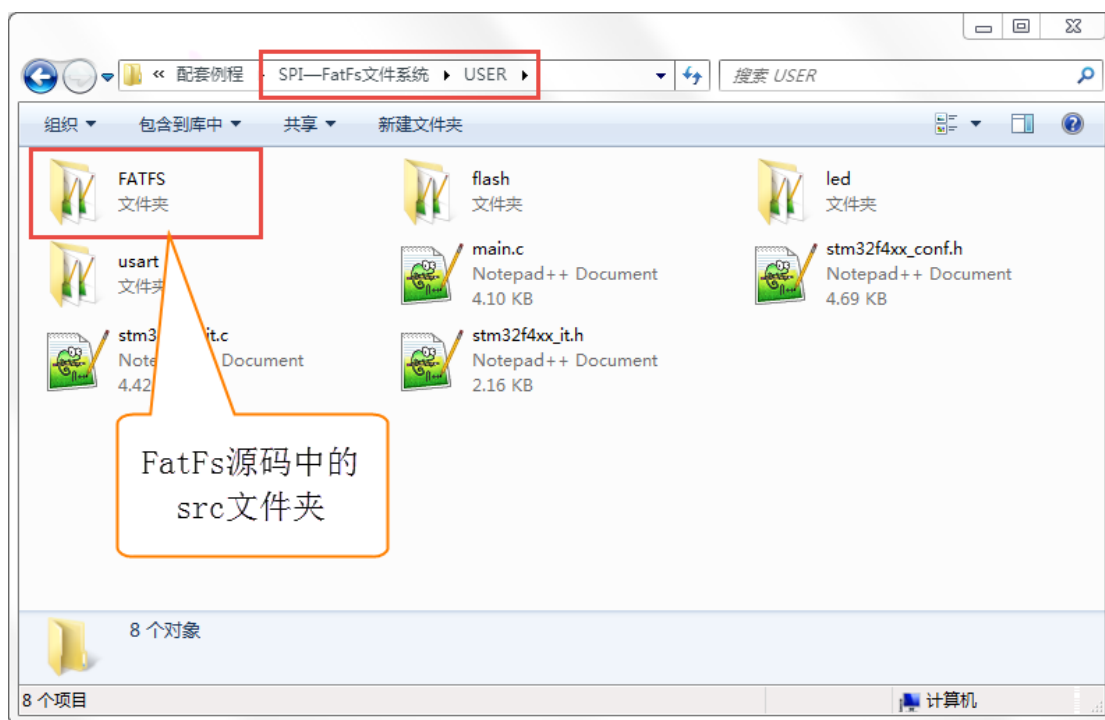


图 26-5 拷贝 FatFs 源码到工程

- 2) 使用 KEIL 软件打开工程文件(.\SPI—FatFs 文件系统\Project\RVMDK(uv5)\BH-F407.uvprojx)，并将 FatFs 组件文件添加到工程中，需要添加有 ff.c、diskio.c 和 cc936.c 三个文件，见图 26-6。



图 26-6 添加 FatFS 文件到工程

- 3) 添加 FATFS 文件夹到工程的 include 选项中。打开工程选项对话框，选择“C/C++”选项下的“Include Paths”项目，在弹出路径设置对话框中选择添加“FATFS”文件夹，见图 26-7。

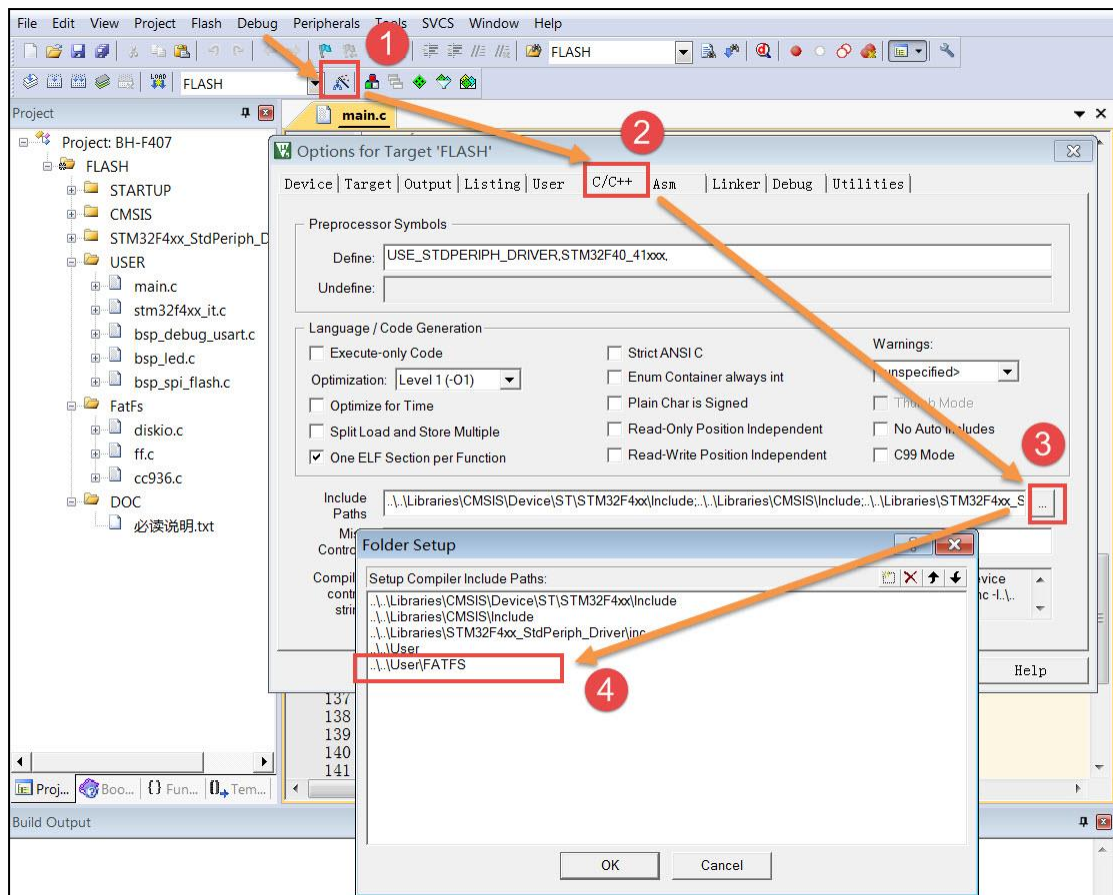


图 26-7 添加 FATFS 路径到工程选项

- 4) 如果现在编译工程，可以发现有两个错误，一个是来自 `diskio.c` 文件，提示有一些头文件没找，`diskio.c` 文件内容是底层设备输入输出接口函数文件，不同硬件设计驱动就不同，需要的文件也不同；另外一个错误来自 `cc936.c` 文件，提示该文件不是工程所必需的，这是因为 FatFs 默认使用日语，我们想要支持简体中文需要修改 FatFs 的配置，即修改 `ffconf.h` 文件。至此，将 FatFs 添加到工程的框架已经操作完成，接下来要做的就是修改 `diskio.c` 文件和 `ffconf.h` 文件。

26.3.4 FatFs 底层设备驱动函数

FatFs 文件系统与底层介质的驱动分离开来，对底层介质的操作都要交给用户去实现，它仅仅是提供了一个函数接口而已。表 26-1 为 FatFs 移植时用户必须支持的函数。通过表 26-1 我们可以清晰知道很多函数是在一定条件下才需要添加的，只有前三个函数是必须添加的。我们完全可以根据实际需求选择实现用到的函数。

前三个函数是实现读文件最基本需求。接下来三个函数是实现创建文件、修改文件需要的。为实现格式化功能，需要在 `disk_ioctl` 添加两个获取物理设备信息选项。我们一般只有实现前面六个函数就可以了，已经足够满足大部分功能。

为支持简体中文长文件名称需要添加 `ff_convert` 和 `ff_wtoupper` 函数，实际这两个已经在 `cc936.c` 文件中实现了，我们只要直接把 `cc936.c` 文件添加到工程中就可以了。

后面六个函数一般都不需要。如真有需要可以参考 syscall.c 文件(src\option 文件夹内)。

表 26-1 FatFs 移植需要用户支持函数

函数	条件(ffconf.h)	备注
disk_status disk_initialize disk_read	总是需要	底层设备驱动函数
disk_write get_fattime disk_ioctl (CTRL_SYNC)	_FS_READONLY == 0	
disk_ioctl (GET_SECTOR_COUNT) disk_ioctl (GET_BLOCK_SIZE)	_USE_MKFS == 1	
disk_ioctl (GET_SECTOR_SIZE)	_MAX_SS != _MIN_SS	
disk_ioctl (CTRL_TRIM)	_USE_TRIM == 1	
ff_convert ff_wtoupper	_USE_LFN != 0	Unicode 支持，为支持简体中文，添加 cc936.c 到工程即可
ff_cre_syncobj ff_del_syncobj ff_req_grant ff_rel_grant	_FS_REENTRANT == 1	FatFs 可重入配置，需要多任务系统支持(一般不需要)
ff_mem_alloc ff_mem_free	_USE_LFN == 3	长文件名支持，缓冲区设置在堆空间(一般设置 _USE_LFN = 2)

底层设备驱动函数是存放在 diskio.c 文件，我们的目的就是要把 diskio.c 中的函数接口与 SPI Flash 芯片驱动连接起来。总共有五个函数，分别为设备状态获取(disk_status)、设备初始化(disk_initialize)、扇区读取(disk_read)、扇区写入(disk_write)、其他控制(disk_ioctl)。

接下来，我们对每个函数结合 SPI Flash 芯片驱动做详细讲解。

宏定义

代码清单 26-1 物理编号宏定义

```
1 /* 为每个设备定义一个物理编号 */
2 #define ATA          0    // 预留 SD 卡使用
3 #define SPI_FLASH    1    // 外部 SPI Flash
```

这两个宏定义在 FatFs 中非常重要，FatFs 是支持多物理设备的，必须为每个物理设备定义一个不同的编号。

SD 卡是预留接口，在讲解 SDIO 接口相关章节后会用到，可以实现使用读写 SD 卡内文件。

设备状态获取

代码清单 26-2 设备状态获取

```
1 DSTATUS disk_status (
2     BYTE pdrv    /* 物理编号 */
```

```
3 )
4 {
5     DSTATUS status = STA_NOINIT;
6
7     switch (pdrv) {
8     case ATA: /* SD CARD */
9         break;
10
11     case SPI_FLASH:
12         /* SPI Flash 状态检测: 读取 SPI Flash 设备 ID */
13         if (sFLASH_ID == SPI_FLASH_ReadID()) {
14             /* 设备 ID 读取结果正确 */
15             status &= ~STA_NOINIT;
16         } else {
17             /* 设备 ID 读取结果错误 */
18             status = STA_NOINIT;;
19         }
20     }
21     break;
22
23     default:
24         status = STA_NOINIT;
25     }
26     return status;
27 }
```

disk_status 函数只有一个参数 pdrv，表示物理编号。一般我们都是使用 switch 函数实现对 pdrv 的分支判断。对于 SD 卡只是预留接口，留空即可。对于 SPI Flash 芯片，我们直接调用在 SPI_FLASH_ReadID() 获取设备 ID，然后判断是否正确，如果正确，函数返回正常标准；如果错误，函数返回异常标志。SPI_FLASH_ReadID() 是定义在 bsp_spi_flash.c 文件中，上一章节已做了分析。

设备初始化

代码清单 26-3 设备初始化

```
1 DSTATUS disk_initialize (
2     BYTE pdrv /* 物理编号 */
3 )
4 {
5     uint16_t i;
6     DSTATUS status = STA_NOINIT;
7     switch (pdrv) {
8     case ATA: /* SD CARD */
9         break;
10
11     case SPI_FLASH: /* SPI Flash */
12         /* 初始化 SPI Flash */
13         SPI_FLASH_Init();
14         /* 延时一小段时间 */
15         i=500;
16         while (--i);
17         /* 唤醒 SPI Flash */
18         SPI_Flash_WAKEUP();
19         /* 获取 SPI Flash 芯片状态 */
20         status=disk_status(SPI_FLASH);
21         break;
22
23     default:
24         status = STA_NOINIT;
25     }
26     return status;
27 }
```

disk_initialize 函数也是有一个参数 pdrv，用来指定设备物理编号。对于 SPI Flash 芯片我们调用 SPI_FLASH_Init()函数实现对 SPI Flash 芯片引脚 GPIO 初始化配置以及 SPI 通信参数配置。SPI_Flash_WAKEUP()函数唤醒 SPI Flash 芯片，当 SPI Flash 芯片处于睡眠模式时需要唤醒芯片才可以进行读写操作。

最后调用 disk_status 函数获取 SPI Flash 芯片状态，并返回状态值。

读取扇区

代码清单 26-4 扇区读取

```
1 DRESULT disk_read (
2     BYTE pdrv,      /* 设备物理编号 (0..) */
3     BYTE *buff,     /* 数据缓存区 */
4     DWORD sector,   /* 扇区首地址 */
5     UINT count      /* 扇区个数 (1..128) */
6 )
7 {
8     DRESULT status = RES_PARERR;
9     switch (pdrv) {
10    case ATA: /* SD CARD */
11        break;
12
13    case SPI_FLASH:
14        /* 扇区偏移 6MB，外部 Flash 文件系统空间放在 SPI Flash 后面 10MB 空间 */
15        sector+=1536;
16        SPI_FLASH_BufferRead(buff, sector <<12, count<<12);
17        status = RES_OK;
18        break;
19
20    default:
21        status = RES_PARERR;
22    }
23    return status;
24 }
```

disk_read 函数有四个形参。pdrv 为设备物理编号。buff 是一个 BYTE 类型指针变量，buff 指向用来存放读取到数据的存储区首地址。sector 是一个 DWORD 类型变量，指定要读取数据的扇区首地址。count 是一个 UINT 类型变量，指定扇区数量。

BYTE 类型实际是 unsigned char 类型，DWORD 类型实际是 unsigned long 类型，UINT 类型实际是 unsigned int 类型，类型定义在 integer.h 文件中。

开发板使用的 SPI Flash 芯片型号为 W25Q128FV，每个扇区大小为 4096 个字节(4KB)，总共有 16M 字节空间，为兼容后面实验程序，我们只将后部分 10MB 空间分配给 FatFs 使用，前部分 6MB 空间用于其他实验需要，即 FatFs 是从 6MB 空间开始，为实现这个效果需要将所有的读写地址都偏移 1536 个扇区空间。

对于 SPI Flash 芯片，主要是使用 SPI_FLASH_BufferRead()实现在指定地址读取指定长度的数据，它接收三个参数，第一个参数为指定数据存放地址指针。第二个参数为指定数据读取地址，这里使用左移运算符，左移 12 位实际是乘以 4096，这与每个扇区大小是息息相关的。第三个参数为读取数据个数，也是需要使用左移运算符。

扇区写入

代码清单 26-5 扇区输入

```
1 DRESULT disk_write (
2     BYTE pdrv,          /* 设备物理编号(0..) */
3     const BYTE *buff,   /* 欲写入数据的缓存区 */
4     DWORD sector,       /* 扇区首地址 */
5     UINT count          /* 扇区个数(1..128) */
6 )
7 {
8     uint32_t write_addr;
9     DRESULT status = RES_PARERR;
10    if (!count) {
11        return RES_PARERR;    /* Check parameter */
12    }
13
14    switch (pdrv) {
15        case ATA: /* SD CARD */
16            break;
17
18        case SPI_FLASH:
19            /* 扇区偏移 6MB, 外部 Flash 文件系统空间放在 SPI Flash 后面 10MB 空间 */
20            sector+=1536;
21            write_addr = sector<<12;
22            SPI_FLASH_SectorErase(write_addr);
23            SPI_FLASH_BufferWrite((u8 *)buff,write_addr,count<<12);
24            status = RES_OK;
25            break;
26
27        default:
28            status = RES_PARERR;
29    }
30    return status;
31 }
```

disk_write 函数有四个形参, pdrv 为设备物理编号。buff 指向待写入扇区数据的首地址。sector, 指定要读取数据的扇区首地址。count 指定扇区数量。对于 SPI Flash 芯片, 在写入数据之前需要先擦除, 所以用到扇区擦除函数(SPI_FLASH_SectorErase)。然后就是在调用数据写入函数(SPI_FLASH_BufferWrite)把数据写入到指定位置内。

其他控制

代码清单 26-6 其他控制

```
1 DRESULT disk_ioctl (
2     BYTE pdrv,          /* 物理编号 */
3     BYTE cmd,           /* 控制指令 */
4     void *buff          /* 写入或者读取数据地址指针 */
5 )
6 {
7     DRESULT status = RES_PARERR;
8     switch (pdrv) {
9         case ATA: /* SD CARD */
10            break;
11
12        case SPI_FLASH:
13            switch (cmd) {
14                /* 扇区数量: 2560*4096/1024/1024=10 (MB) */
15                case GET_SECTOR_COUNT:
16                    *(DWORD *)buff = 2560;
17                    break;
18                /* 扇区大小 */
19                case GET_SECTOR_SIZE :
20                    *(WORD *)buff = 4096;
21                    break;
22                /* 同时擦除扇区个数 */
23                case GET_BLOCK_SIZE :
```

```
24         *(DWORD *)buff = 1;
25         break;
26     }
27     status = RES_OK;
28     break;
29
30     default:
31         status = RES_PARERR;
32     }
33     return status;
34 }
```

disk_ioctl 函数有三个形参，pdrv 为设备物理编号，cmd 为控制指令，包括发出同步信号、获取扇区数目、获取扇区大小、获取擦除块数量等等指令，buff 为指令对应的数据指针。

对于 SPI Flash 芯片，为支持 FatFs 格式化功能，需要用到获取扇区数量 (GET_SECTOR_COUNT) 指令和获取擦除块数量 (GET_BLOCK_SIZE)。另外，SD 卡扇区大小为 512 字节，SPI Flash 芯片一般设置扇区大小为 4096 字节，所以需要用到获取扇区大小 (GET_SECTOR_SIZE) 指令。

时间戳获取

代码清单 26-7 时间戳获取

```
1  __weak DWORD get_fattime(void)
2  {
3      /* 返回当前时间戳 */
4      return ((DWORD)(2015 - 1980) << 25) /* Year 2015 */
5          | ((DWORD)1 << 21) /* Month 1 */
6          | ((DWORD)1 << 16) /* Mday 1 */
7          | ((DWORD)0 << 11) /* Hour 0 */
8          | ((DWORD)0 << 5) /* Min 0 */
9          | ((DWORD)0 >> 1); /* Sec 0 */
10 }
```

get_fattime 函数用于获取当前时间戳，在 ff.c 文件中被调用。FatFs 在文件创建、被修改时会记录时间，这里我们直接使用赋值方法设定时间戳。为更好的记录时间，可以使用控制器 RTC 功能，具体要求返回值格式为：

- ❑ bit31:25 ——从 1980 至今是多少年，范围是 (0..127)；
- ❑ bit24:21 ——月份，范围为 (1..12)；
- ❑ bit20:16 ——该月份中的第几日，范围为(1..31)；
- ❑ bit15:11 ——时，范围为 (0..23)；
- ❑ bit10:5 ——分，范围为 (0..59)；
- ❑ bit4:0 ——秒/2，范围为 (0..29)。

26.3.5 FatFs 功能配置

ffconf.h 文件是 FatFs 功能配置文件，我们可以对文件内容进行修改，使得 FatFs 更符合我们的要求。ffconf.h 对每个配置选项都做了详细的使用情况说明。下面只列出修改的配置，其他配置采用默认即可。

代码清单 26-8 FatFs 功能配置选项

```
1 #define _USE_MKFS 1
2 #define _CODE_PAGE 936
3 #define _USE_LFN 2
4 #define _VOLUMES 2
5 #define _MIN_SS 512
6 #define _MAX_SS 4096
```

- (1) `_USE_MKFS`: 格式化功能选择, 为使用 FatFs 格式化功能, 需要把它设置为 1。
- (2) `_CODE_PAGE`: 语言功能选择, 并要求把相关语言文件添加到工程宏。为支持简体中文文件名需要使用“936”, 正如在图 26-6 的操作, 我们已经把 `cc936.c` 文件添加到工程中。
- (3) `_USE_LFN`: 长文件名支持, 默认不支持长文件名, 这里配置为 2, 支持长文件名, 并指定使用栈空间为缓冲区。
- (4) `_VOLUMES`: 指定物理设备数量, 这里设置为 2, 包括预留 SD 卡和 SPI Flash 芯片。
- (5) `_MIN_SS`、`_MAX_SS`: 指定扇区大小的最小值和最大值。SD 卡扇区大小一般都为 512 字节, SPI Flash 芯片扇区大小一般设置为 4096 字节, 所以需要把 `_MAX_SS` 改为 4096。

26.3.6 FatFs 功能测试

移植操作到此, 就已经把 FatFs 全部添加到我们的工程了, 这时我们编译功能, 顺利编译通过, 没有错误。接下来, 我们就可以使用编写图 26-4 中用户应用程序了。

主要的测试包括格式化测试、文件写入测试和文件读取测试三个部分, 主要程序都在 `main.c` 文件中实现。

变量定义

代码清单 26-9 变量定义

```
1 FATFS fs; /* FatFs 文件系统对象 */
2 FIL fnew; /* 文件对象 */
3 FRESULT res_flash; /* 文件操作结果 */
4 UINT fnum; /* 文件成功读写数量 */
5 BYTE buffer[1024] = {0}; /* 读缓冲区 */
6 BYTE textFileBuffer[] = /* 写缓冲区 */
7 "欢迎使用野火 STM32 F407 开发板 今天是个好日子, 新建文件系统测试文件\r\n";
```

FATFS 是在 `ff.h` 文件定义的一个结构体类型, 针对的对象是物理设备, 包含了物理设备的物理编号、扇区大小等信息, 一般我们都需要为每个物理设备定义一个 FATFS 变量。

FIL 也是在 `ff.h` 文件定义的一个结构体类型, 针对的对象是文件系统内具体的文件, 包含了文件很多基本属性, 比如文件大小、路径、当前读写地址等等。如果需要在同一时间打开多个文件进行读写, 才需要定义多个 FIL 变量, 不然一般定义一个 FIL 变量即可。

FRESULT 是也在 `ff.h` 文件定义的一个枚举类型, 作为 FatFs 函数的返回值类型, 主要管理 FatFs 运行中出现的错误。总共有 19 种错误类型, 包括物理设备读写错误、找不到文件、没有挂载工作空间等等错误。这在实际编程中非常重要, 当有错误出现是我们要停止文件读写, 通过返回值我们可以快速定位到错误发生的可能地点。如果运行没有错误才返回 `FR_OK`。

`fnum` 是个 32 位无符号整形变量, 用来记录实际读取或者写入数据的数组。

buffer 和 textFileBuffer 分别对应读取和写入数据缓存区，都是 8 位无符号整形数组。

主函数

代码清单 26-10 主函数

```
1 int main(void)
2 {
3     /* 初始化 LED */
4     LED_GPIO_Config();
5     LED_BLUE;
6
7     /* 初始化调试串口，一般为串口 1 */
8     Debug_USART_Config();
9     printf("***** 这是一个 SPI FLASH 文件系统实验 *****\r\n");
10
11     //在外部 SPI Flash 挂载文件系统，文件系统挂载时会对 SPI 设备初始化
12     res_flash = f_mount(&fs, "1:", 1);
13
14     /*----- 格式化测试 -----*/
15     /* 如果没有文件系统就格式化创建文件系统 */
16     if (res_flash == FR_NO_FILESYSTEM) {
17         printf("》FLASH 还没有文件系统，即将进行格式化...\r\n");
18         /* 格式化 */
19         res_flash=f_mkfs("1:", 0, 0);
20
21         if (res_flash == FR_OK) {
22             printf("》FLASH 已成功格式化文件系统。 \r\n");
23             /* 格式化后，先取消挂载 */
24             res_flash = f_mount(NULL, "1:", 1);
25             /* 重新挂载 */
26             res_flash = f_mount(&fs, "1:", 1);
27         } else {
28             LED_RED;
29             printf("《《格式化失败。》》 \r\n");
30             while (1);
31         }
32     } else if (res_flash!=FR_OK) {
33         printf("!! 外部 Flash 挂载文件系统失败。( %d)\r\n", res_flash);
34         printf("!! 可能原因: SPI Flash 初始化不成功。 \r\n");
35         while (1);
36     } else {
37         printf("》文件系统挂载成功，可以进行读写测试\r\n");
38     }
39
40     /*----- 文件系统测试: 写测试 -----*/
41     /* 打开文件，如果文件不存在则创建它 */
42     printf("\r\n***** 即将进行文件写入测试... *****\r\n");
43     res_flash = f_open(&fnew, "1:FatFs 读写测试文件.txt",
44                       FA_CREATE_ALWAYS | FA_WRITE );
45     if ( res_flash == FR_OK ) {
46         printf("》打开/创建 FatFs 读写测试文件.txt 文件成功，向文件写入数据。 \r\n");
47         /* 将指定存储区内容写入到文件内 */
48         res_flash=f_write(&fnew, WriteBuffer, sizeof(WriteBuffer), &fnum);
49         if (res_flash==FR_OK) {
50             printf("》文件写入成功，写入字节数据: %d\n", fnum);
51             printf("》向文件写入的数据为: \r\n%s\r\n", WriteBuffer);
52         } else {
53             printf("!! 文件写入失败: (%d)\n", res_flash);
54         }
55         /* 不再读写，关闭文件 */
56         f_close(&fnew);
57     } else {
```

```
58     LED_RED;
59     printf("!! 打开/创建文件失败。\\r\\n");
60 }
61
62 /*----- 文件系统测试: 读测试 -----*/
63 printf("***** 即将进行文件读取测试... *****\\r\\n");
64 res_flash = f_open(&fnew, "1:FatFs 读写测试文件.txt",
65                   FA_OPEN_EXISTING | FA_READ);
66 if (res_flash == FR_OK) {
67     LED_GREEN;
68     printf("》 打开文件成功。\\r\\n");
69     res_flash = f_read(&fnew, ReadBuffer, sizeof(ReadBuffer), &fnum);
70     if (res_flash==FR_OK) {
71         printf("》 文件读取成功,读到字节数据: %d\\r\\n",fnum);
72         printf("》 读取得的文件数据为: \\r\\n%s \\r\\n", ReadBuffer);
73     } else {
74         printf("!! 文件读取失败: (%d)\\n",res_flash);
75     }
76 } else {
77     LED_RED;
78     printf("!! 打开文件失败。\\r\\n");
79 }
80 /* 不再读写, 关闭文件 */
81 f_close(&fnew);
82
83 /* 不再使用文件系统, 取消挂载文件系统 */
84 f_mount(NULL, "1:", 1);
85
86 /* 操作完成, 停机 */
87 while (1) {
88 }
89 }
```

首先, 初始化 RGB 彩灯和调试串口, 用来指示程序进程。

FatFs 的第一步工作就是使用 `f_mount` 函数挂载工作区。`f_mount` 函数有三个形参, 第一个参数是指向 FATFS 变量指针, 如果赋值为 `NULL` 可以取消物理设备挂载。第二个参数为逻辑设备编号, 使用设备根路径表示, 与物理设备编号挂钩, 在代码清单 26-1 中我们定义 SPI Flash 芯片物理编号为 1, 所以这里使用“1: ”。第三个参数可选 0 或 1, 1 表示立即挂载, 0 表示不立即挂载, 延迟挂载。`f_mount` 函数会返回一个 `FRESULT` 类型值, 指示运行情况。

如果 `f_mount` 函数返回值为 `FR_NO_FILESYSTEM`, 说明没有 FAT 文件系统, 比如新出厂的 SPI Flash 芯片就没有 FAT 文件系统。我们就必须对物理设备进行格式化处理。使用 `f_mkfs` 函数可以实现格式化操作。`f_mkfs` 函数有三个形参, 第一个参数为逻辑设备编号; 第二参数可选 0 或者 1, 0 表示设备为一般硬盘, 1 表示设备为软盘。第三个参数指定扇区大小, 如果为 0, 表示通过代码清单 26-6 中 `disk_ioctl` 函数获取。格式化成功后需要先取消挂载原来设备, 再重新挂载设备。

在设备正常挂载后, 就可以进行文件读写操作了。使用文件之前, 必须使用 `f_open` 函数打开文件, 不再使用文件必须使用 `f_close` 函数关闭文件, 这个跟电脑端操作文件步骤类似。`f_open` 函数有三个形参, 第一个参数为文件对象指针。第二参数为目标文件, 包含绝对路径的文件名称和后缀名。第三个参数为访问文件模式选择, 可以是打开已经存在的文件模式、读模式、写模式、新建模式、总是新建模式等的或运行结果。比如对于写测试, 使用 `FA_CREATE_ALWAYS` 和 `FA_WRITE` 组合模式, 就是总是新建文件并进行写模式。

`f_close` 函数用于不再对文件进行读写操作关闭文件，`f_close` 函数只要一个形参，为文件对象指针。`f_close` 函数运行可以确保缓冲区完全写入到文件内。

成功打开文件之后就可以使用 `f_write` 函数和 `f_read` 函数对文件进行写操作和读操作。这两个函数用到的参数是一致的，只不过一个是数据写入，一个是数据读取。`f_write` 函数第一个形参为文件对象指针，使用与 `f_open` 函数一致即可。第二个参数为待写入数据的首地址，对于 `f_read` 函数就是用来存放读出数据的首地址。第三个参数为写入数据的字节数，对于 `f_read` 函数就是欲读取数据的字节数。第四个参数为 32 位无符号整形指针，这里使用 `fnum` 变量地址赋值给它，在运行读写操作函数后，`fnum` 变量指示成功读取或者写入的字节个数。

最后，不再使用文件系统时，使用 `f_unmount` 函数取消挂载。

26.3.7 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。程序开始运行后，RGB 彩灯为蓝色，在串口调试助手可看到格式化测试、写文件检测和读文件检测三个过程；最后如果所有读写操作都正常，RGB 彩灯会指示为绿色，如果在运行中 FatFs 出现错误 RGB 彩灯指示为红色。

虽然我们通过 RGB 彩灯指示和串口调试助手信息打印方法来说明 FatFs 移植成功，并顺利通过测试，但心底总是不踏实，所谓眼见为实，虽然我们创建了“FatFs 读写测试文件.txt”这个文件，却完全看不到实体。这个确实是个问题，因为我们这里使用 SPI Flash 芯片作为物理设备，并不像 SD 卡那么方便直接用读卡器就可以在电脑端打开验证。另外一个问题，就目前来说，在 SPI Flash 芯片上挂载 FatFs 好像没有实际意义，无法发挥文件系统功能。

实际上，这里归根到底就是我们目前没办法在电脑端查看 SPI Flash 芯片内 FatFs 的内容，没办法非常方便拷贝、删除文件。我们当然不会做无用功，STM32 控制器还有一个硬件资源可以解决上面的问题，就是 USB！我们可以通过编程把整个开发板变成一个 U 盘，而 U 盘存储空间就是 SPI Flash 芯片的空间。这样非常方便实现文件读写。至于 USB 内容将在 USB 相关章节讲解。

26.4 FatFs 功能使用实验

上个实验我们实现了 FatFs 的格式化、读文件和写文件功能，这个已经满足很多部分的运用需要。有时，我们需要更多的文件操作功能，FatFs 还是提供了不少的功能的，比如设备存储空间信息获取、读写文件指针定位、创建目录、文件移动和重命名、文件或目录信息获取等等功能。我们接下来这个实验内容就是展示 FatFs 众多功能，提供一个很好了范例，以后有用到相关内容，参考使用非常方便。

26.4.1 硬件设计

本实验主要使用 FatFs 软件功能，不需要其他硬件模块，使用与 FatFs 移植实验相同硬件配置即可。

26.4.2 软件设计

上个实验我们已经移植好了 FatFs，这个例程主要是应用，所以简单起见，直接拷贝上个实验的工程文件，保持 FatFs 底层驱动程序，我们只改 main.c 文件内容，实现应用程序。

FatFs 多项功能测试

代码清单 26-11 FatFs 多项功能测试

```
1 static FRESULT miscellaneous(void)
2 {
3     DIR dir;
4     FATFS *pfs;
5     DWORD fre_clust, fre_sect, tot_sect;
6
7     printf("\n***** 设备信息获取 *****\r\n");
8     /* 获取设备信息和空簇大小 */
9     res_flash = f_getfree("1:", &fre_clust, &pfs);
10
11     /* 计算得到总的扇区个数和空扇区个数 */
12     tot_sect = (pfs->n_fatent - 2) * pfs->csize;
13     fre_sect = fre_clust * pfs->csize;
14
15     /* 打印信息(4096 字节/扇区) */
16     printf("》设备总空间: %10lu KB.\n》可用空间: %10lu KB.\n",
17           tot_sect * 4, fre_sect * 4);
18
19     printf("\n***** 文件定位和格式化写入功能测试 *****\r\n");
20     res_flash = f_open(&fnew, "1:FatFs 读写测试文件.txt",
21                       FA_OPEN_EXISTING|FA_WRITE|FA_READ );
22     if ( res_flash == FR_OK ) {
23         /* 文件定位 */
24         res_flash = f_lseek(&fnew,f_size(&fnew)-1);
25         if (res_flash == FR_OK) {
26             /* 格式化写入, 参数格式类似 printf 函数 */
27             f_printf(&fnew,"\n 在原来文件新添加一行内容\n");
28             f_printf(&fnew,"》设备总空间: %10lu KB.\n》可用空间: %10lu KB.\n",
29                     tot_sect * 4, fre_sect * 4);
30
31             /* 文件定位到文件起始位置 */
32             res_flash = f_lseek(&fnew,0);
33             /* 读取文件所有内容到缓存区 */
34             res_flash = f_read(&fnew,readbuffer,f_size(&fnew),&fnum);
35             if (res_flash == FR_OK) {
36                 printf("》文件内容: %s\n",readbuffer);
37             }
38         }
39         f_close(&fnew);
40
41         printf("\n***** 目录创建和重命名功能测试 *****\r\n");
42         /* 尝试打开目录 */
43         res_flash=f_opendir(&dir,"1:TestDir");
44         if (res_flash!=FR_OK) {
45             /* 打开目录失败, 就创建目录 */
46             res_flash=f_mkdir("1:TestDir");
```

```
46         } else {
47             /* 如果目录已经存在, 关闭它 */
48             res_flash=f_closedir(&dir);
49             /* 删除文件 */
50             f_unlink("1:TestDir/testdir.txt");
51         }
52         if (res_flash==FR_OK) {
53             /* 重命名并移动文件 */
54             res_flash=f_rename("1:FatFs 读写测试文件.txt",
55                               "1:TestDir/testdir.txt");
56         }
57     } else {
58         printf("!! 打开文件失败: %d\n", res_flash);
59         printf("!! 或许需要再次运行“FatFs 移植与读写测试”工程\n");
60     }
61     return res_flash;
62 }
```

首先是设备存储信息获取, 目的是获取设备总容量和剩余可用空间。f_getfree 函数是设备空闲簇信息获取函数, 有三个形参, 第一个参数为逻辑设备编号; 第二个参数为返回空闲簇数量; 第三个参数为返回指向文件系统对象的指针。通过计算可得到设备总的扇区个数以及空闲扇区个数, 对于 SPI Flash 芯片我们设置每个扇区为 4096 字节大小。这样很容易就算出设备存储信息。

接下来是文件读写指针定位和格式化输入功能测试。文件定位在一些场合非常有用, 比如我们需要记录多项数据, 但每项数据长度不确定, 但有个最长长度, 使用我们就可以使用文件定位 lseek 函数功能把数据存放在规定好的地址空间上。当我们需要读取文件内容时就使用文件定位函数定位到对应地址读取。

使用文件读写操作之前都必须使用 f_open 函数打开文件, 开始文件是读写指针是在文件起始位置的, 马上写入数据的话会覆盖原来文件内容的。这里, 我们使用 f_lseek 函数定位到文件末尾位置, 再写入内容。f_lseek 函数有两个形参, 第一个参数为文件对象指针, 第二个参数为需要定位的字节数, 这个字节数是相对文件起始位置的, 比如设置为 0, 则将文件读写指针定位到文件起始位置了。

f_printf 函数是格式化写入函数, 需要把 ffconf.h 文件中的 _USE_STRFUNC 配置为 1 才支持。f_printf 函数用法类似 C 库函数 printf 函数, 只是它将数据直接写入到文件中。

最后是目录创建和文件移动和重命名功能。使用 f_opendir 函数可以打开路径(这里不区分目录和路径概念, 下同), 如果路径不存在则返回错误, 使用 f_closedir 函数关闭已经打开的路径。新版的 FatFs 支持相对路径功能, 使路径操作更加灵活。f_opendir 函数有两个形参, 第一个参数为指向路径对象的指针, 第二个参数为路径。f_closedir 函数只需要指向路径对象的指针一个形参。

f_mkdir 函数用于创建路径, 如果指定的路径不存在就创建它, 创建的路径存在形式就是文件夹。f_mkdir 函数只要一个形参, 就是指定路径。

f_rename 函数是带有移动功能的重命名函数, 它有两个形参, 第一个参数为源文件名称, 第二个参数为目标名称。目标名称可附带路径, 如果路径与源文件路径不同见移动文件到目标路径下。

文件信息获取

代码清单 26-12 文件信息获取

```
1 static FRESULT file_check(void)
2 {
3     FILINFO fno;
4
5     /* 获取文件信息 */
6     res_flash=f_stat("1:TestDir/testdir.txt",&fno);
7     if (res_flash==FR_OK) {
8         printf("\"testdir.txt\"文件信息: \n");
9         printf("》文件大小: %ld(字节)\n", fno.fsize);
10        printf("》时间戳: %u/%02u/%02u, %02u:%02u\n",
11              (fno.fdate >> 9) + 1980, fno.fdate >> 5 & 15, fno.fdate & 31,
12              fno.ftime >> 11, fno.ftime >> 5 & 63);
13        printf("》属性: %c%c%c%c%c\n\n",
14              (fno.fattrib & AM_DIR) ? 'D' : '-', // 是一个目录
15              (fno.fattrib & AM_RDO) ? 'R' : '-', // 只读文件
16              (fno.fattrib & AM_HID) ? 'H' : '-', // 隐藏文件
17              (fno.fattrib & AM_SYS) ? 'S' : '-', // 系统文件
18              (fno.fattrib & AM_ARC) ? 'A' : '-'); // 档案文件
19    }
20    return res_flash;
21 }
```

f_stat 函数用于获取文件的属性，有两个形参，第一个参数为文件路径，第二个参数为返回指向文件信息结构体变量的指针。文件信息结构体变量包含文件的大小、最后修改时间和日期、文件属性、短文件名以及长文件名等信息。

路径扫描

代码清单 26-13 路径扫描

```
1 static FRESULT scan_files (char* path)
2 {
3     FRESULT res; //部分在递归过程被修改的变量，不用全局变量
4     FILINFO fno;
5     DIR dir;
6     int i;
7     char *fn; // 文件名
8
9     #if _USE_LFN
10    /* 长文件名支持 */
11    /* 简体中文需要 2 个字节保存一个“字” */
12    static char lfn[_MAX_LFN*2 + 1];
13    fno.lfname = lfn;
14    fno.lfsize = sizeof(lfn);
15    #endif
16    //打开目录
17    res = f_opendir(&dir, path);
18    if (res == FR_OK) {
19        i = strlen(path);
20        for (;;) {
21            //读取目录下的内容，再读会自动读下一个文件
22            res = f_readdir(&dir, &fno);
23            //为空时表示所有项目读取完毕，跳出
24            if (res != FR_OK || fno.fname[0] == 0) break;
25        #if _USE_LFN
26            fn = *fno.lfname ? fno.lfname : fno.fname;
27        #else
28            fn = fno.fname;
29        #endif
30            //点表示当前目录，跳过
31            if (*fn == '.') continue;
32            //目录，递归读取
33            if (fno.fattrib & AM_DIR) {
```



```
34         //合成完整目录名
35         sprintf(&path[i], "%s", fn);
36         //递归遍历
37         res = scan_files(path);
38         path[i] = 0;
39         //打开失败, 跳出循环
40         if (res != FR_OK)
41             break;
42     } else {
43         printf("%s/%s\r\n", path, fn);           //输出文件名
44         /* 可以在这里提取特定格式的文件路径 */
45     } //else
46 } //for
47 }
48 return res;
49 }
```

scan_files 函数用来扫描指定路径下的文件。比如我们设计一个 mp3 播放器, 我们需要提取 mp3 格式文件, 诸如*.txt、*.c 文件我们统统不可要的, 这时我们就必须扫描路径下所有文件并把*.mp3 或*.MP3 格式文件提取出来。这里我们提取特定格式文件, 而是把所有文件名称都通过串口打印出来。

我们在 ffconf.h 文件中定义了长文件名称支持(_USE_LFN=2), 一般有用到简体中文文件名称的都要长文件名支持。短文件名称是 8.3 格式, 即名称是 8 个字节, 后缀名是 3 个字节, 对于使用英文名称还可以, 使用中文名称就很容易长度不够了。使能了长文件名支持后, 使用之前需要指定文件名的存储区还有存储区的大小。

接下来就是使用 f_opendir 函数打开指定的路径。如果路径存在就使用 f_readdir 函数读取路径下内容, f_readdir 函数可以读取路径下的文件或者文件夹, 并保存信息到文件信息对象变量内。f_readdir 函数有两个形参, 第一个参数为指向路径对象变量的指针, 第二个参数为指向文件信息对象的指针。f_readdir 函数另外一个特性是自动读取下一个文件对象, 即循序运行该函数可以读取该路径下的所有文件。所以, 在程序中, 我们使用 for 循环让 f_readdir 函数读取所有文件, 并在读取所有文件之后退出循环。

在 f_readdir 函数成功读取到一个对象时, 我们还不清楚它是一个文件还是一个文件夹, 此时我们就可以使用文件信息对象变量的文件属性来判断了, 如果判断得出是个文件那我们就直接通过串口打印出来就好了。如果是文件夹, 我们就要进入该文件夹扫描, 这时就重新调用扫描函数 scan_files 就可以了, 形成一个递归调用结构, 只是我们这次用的参数与最开始时候是不同的, 现在是使用子文件夹名称。

主函数

代码清单 26-14 主函数

```
1 int main(void)
2 {
3     /* 初始化调试串口, 一般为串口 1 */
4     Debug_USART_Config();
5     printf("***** 这是一个 SPI FLASH 文件系统实验 *****\r\n");
6
7     //在外部 SPI Flash 挂载文件系统, 文件系统挂载时会对 SPI 设备初始化
8     res_flash = f_mount(&fs, "1:", 1);
9     if (res_flash != FR_OK) {
10         printf("!! 外部 Flash 挂载文件系统失败。(%d)\r\n", res_flash);
11         printf("!! 可能原因: SPI Flash 初始化不成功。 \r\n");
12     }
```

```
12     while (1);
13 } else {
14     printf("》文件系统挂载成功，可以进行测试\r\n");
15 }
16
17 /* FatFs 多项功能测试 */
18 res_flash = miscellaneous();
19
20
21 printf("\n***** 文件信息获取测试 *****\r\n");
22 res_flash = file_check();
23
24
25 printf("***** 文件扫描测试 *****\r\n");
26 strcpy(fpath, "1:");
27 scan_files(fpath);
28
29
30 /* 不再使用文件系统，取消挂载文件系统 */
31 f_mount(NULL, "1:", 1);
32
33 /* 操作完成，停机 */
34 while (1) {
35 }
36 }
```

串口在程序调试中经常使用，可以把变量值直观打印到串口调试助手，这个信息非常重要，同样在使用之前需要调用 `Debug_USART_Config` 函数完成调试串口初始化。

使用 FatFs 进行文件操作之前都使用 `f_mount` 函数挂载物理设备，这里我们使用 SPI Flash 芯片上的 FAT 文件系统。

接下来我们直接调用 `miscellaneous` 函数进行 FatFs 设备信息获取、文件定位和格式化写入功能以及目录创建和重命名功能测试。调用 `file_check` 函数进行文件信息获取测试。

`scan_files` 函数用来扫描路径下的所有文件，`fpath` 是我们定义的一个包含 100 个元素的字符型数组，并将其赋值为 SPI Flash 芯片物理编号对于的根目录。这样允许 `scan_files` 函数打印 SPI Flash 芯片内 FatFs 所有文件到串口调试助手。注意，这里的定义 `fpaht` 数组是必不可少的，因为 `scan_files` 函数本身是个递归函数，要求实际参数有较大空间的缓存区。

26.4.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。程序开始运行，在串口调试助手可看到每个阶段测试相关信息情况。

